Assignment 2 Guide – Regression with PyTorch

1. What the assignment asks

Exercise 1: A small neural network (64–32–1) that predicts the toxicity of chemical compounds to fish (LC50). 70/10/20 split, training with SGD, early stopping with patience = 10, evaluation with MSE, MAE, R^2 and Pearson, plus two plots.

Exercise 2: On the California Housing data, put an activation after the last layer so that no negative price is predicted (Softplus or exp). Report the code change and the test MSE.

Extra (optional): Also encode the 500k $ upper limit. We did this with `5·sigmoid`.

2. Main results

Exercise 1 (test set, 182 compounds): MSE = 0.862, RMSE = 0.93, MAE = 0.687, R^2 = 0.644, Pearson r = 0.811. Baseline (always the mean): MSE = 2.42, R^2 = 0. Best epoch = 23, stopped at epoch 33.

Exercise 2 (test MSE): no constraint 0.290 (one negative prediction, 37 above 5), Softplus 0.293 (no negatives), exp 0.308, 5·sigmoid 0.291 (no negatives, none above 5).

Main message: the constraint barely changes accuracy, but it removes impossible predictions.

3. Exercise 1, step by step

Step 1 – Load the data

Downloaded with `fetch_openml`: 908 compounds, 6 numeric features, no missing values. LC50 has mean 4.06, standard deviation 1.46, range 0.05 to 9.61. The distribution is roughly bell-shaped with a slight tail to the right, so a few compounds are very toxic.

Features: `MLOGP` = hydrophobicity; `CIC0`, `SM1_Dz`, `GATS1i` = structural and electronic descriptors of the molecule; `NdsCH` and `NdssC` = atom-type counts, i.e. the number of atoms of a certain type (description from the UCI page). These two columns are integers and are 0 for most compounds.

The features have different scales (MLOGP from −2.9 to 6.5, SM1_Dz from 0 to 2.2), so scaling is needed. The dataset is small, so a small network and early stopping make sense. This is the answer to "Describe your findings" in the notebook.

Step 2 – Split and scaling

First 20 % for test, then 12.5 % of the remaining 80 % for validation (0.125 × 0.8 = 0.10). The output `train: (635, 6) val: (91, 6) test: (182, 6)` means (rows, columns): 635 compounds with 6 features, and so on. Total: 635 + 91 + 182 = 908. The target LC50 is not one of these 6 columns; it is kept separately in y.

StandardScaler: computes z = (x − mean) / std for each column. After that every feature has mean 0 and standard deviation 1. The output `means [-0. ...]` and `stds [1. ...]` confirms this. "-0." is a tiny negative number rounded to zero.

Why is it needed? With different scales, the large features dominate the gradient and training becomes slow and unstable.

The scaler is **fitted** only on train and only **transforms** val and test. Otherwise information from the test set leaks into training (data leakage). That is why the mean of val and test is not exactly 0, which is correct. We did not scale y, so that the MSE is in the original unit of LC50.

Step 3 – Tensors and DataLoader

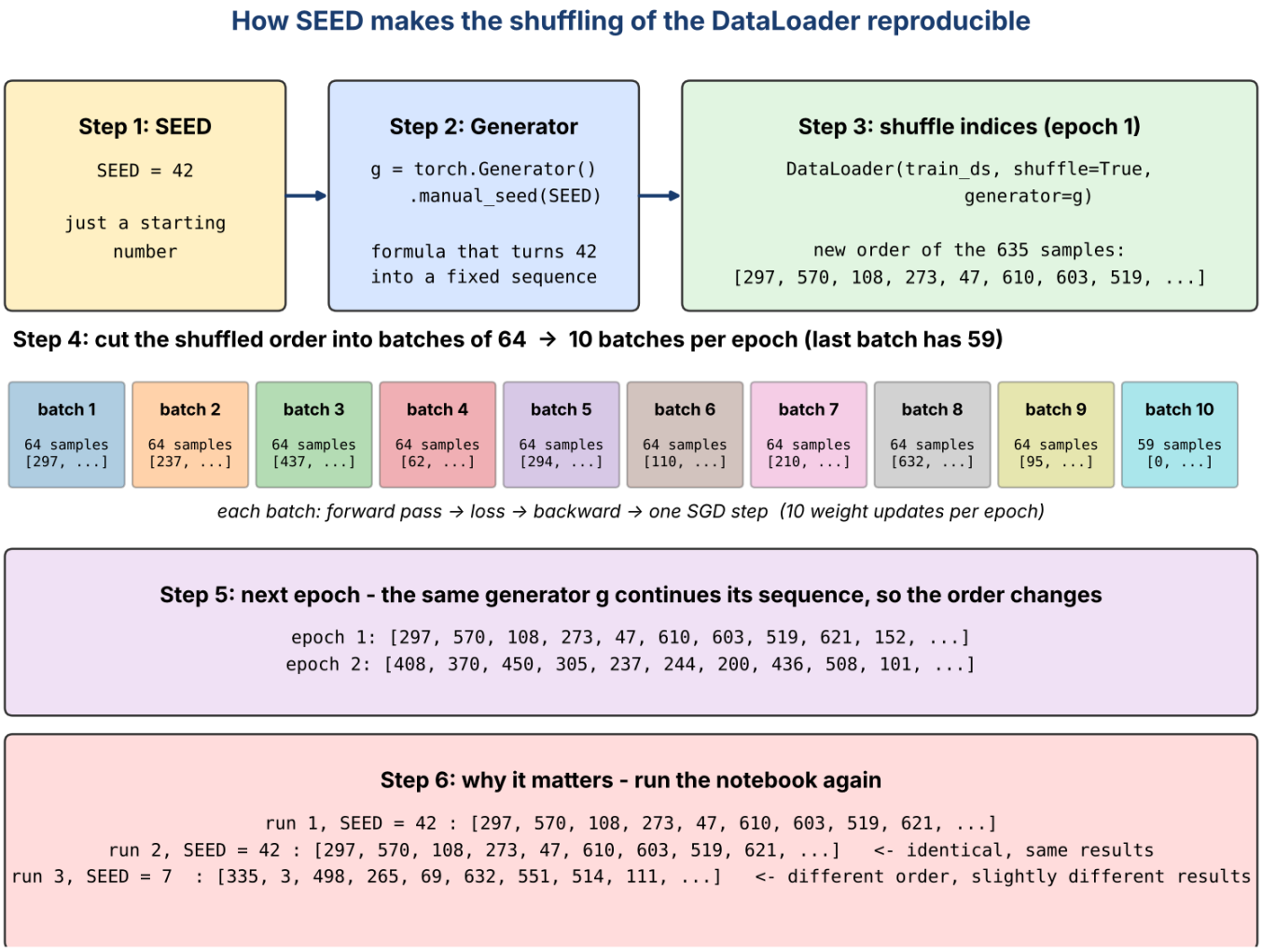
torch is the PyTorch library. A **tensor** is a multi-dimensional array, like a NumPy array, on which PyTorch can compute gradients automatically and which can run on a GPU. Names by number of dimensions: 0 dimensions = a single number (like the loss), 1 = a vector, 2 = a matrix like X_train.

```
X_t = torch.tensor(X_arr, dtype=torch.float32)
y_t = torch.tensor(y_arr, dtype=torch.float32).view(-1, 1)
```

float32 is needed because the network weights are float32 (NumPy creates float64 by default). `.view(-1, 1)` changes y from shape (635,) to (635, 1) so that it matches the network output; otherwise MSELoss is computed incorrectly.

Batch size = 64: required by the sheet. The network updates its weights after every batch: 64 compounds go in together, the loss is their average, and one SGD step is taken. 635 ÷ 64 ≈ 10 batches per epoch (the last one has 59). The output `torch.Size([64, 6])` and `([64, 1])` is the shape of one batch. Very small batches give noisy gradients, very large batches give few steps; 64 is a common, balanced value. This 64 has nothing to do with the 64 neurons of the first layer.

Seed and Generator: `g = torch.Generator().manual_seed(SEED)` creates a random number generator with a fixed start (42). The train_loader with `shuffle=True, generator=g` uses it to shuffle the samples in every epoch. Computer random numbers are pseudo-random: the same seed gives the same sequence, so every run produces the same batches and the same result. In each epoch the generator continues its sequence, so the order differs between epochs but repeats between runs. Val and test are not shuffled.



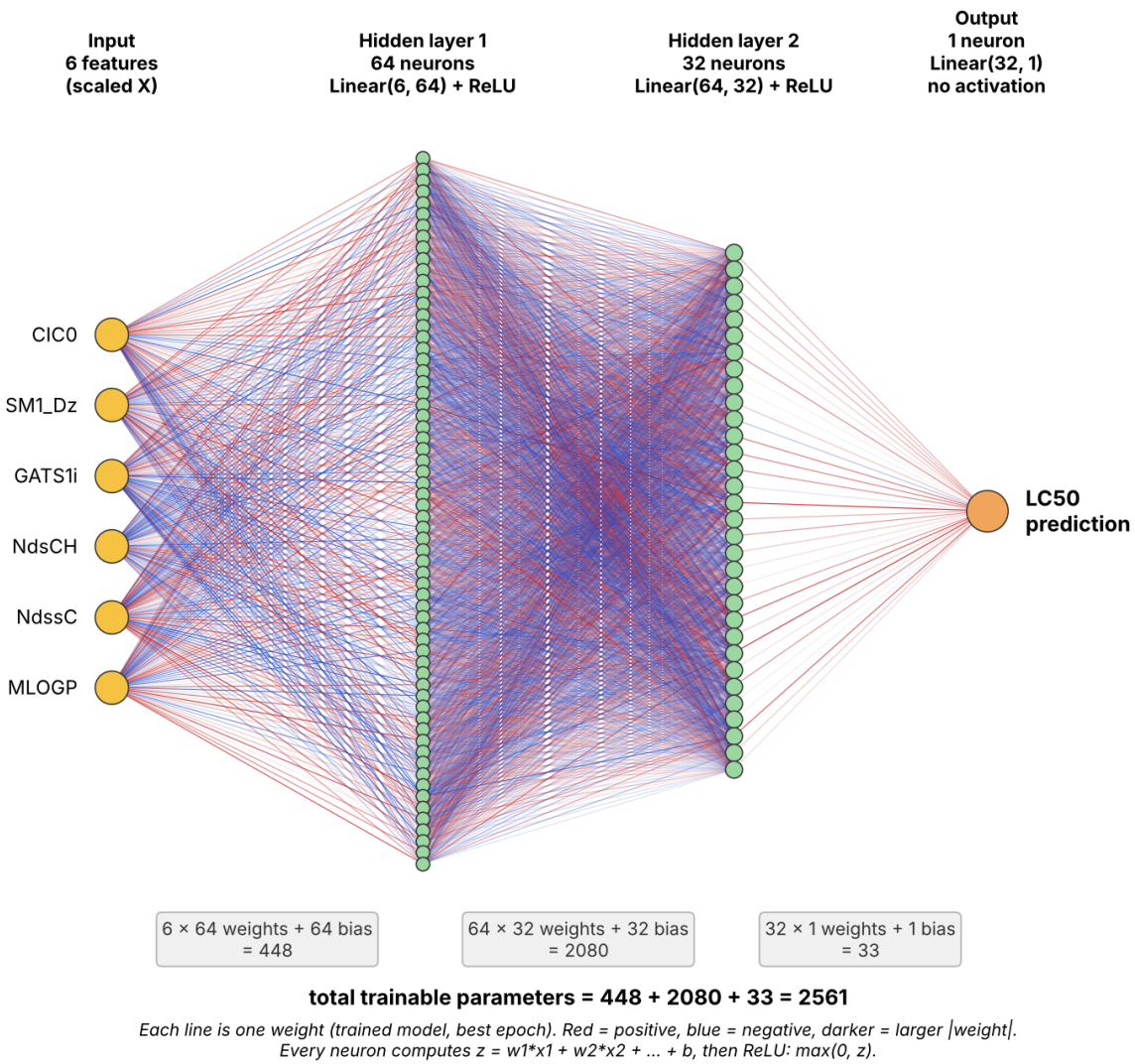
Step 4 – The network

```
Linear(6, 64) → ReLU → Linear(64, 32) → ReLU → Linear(32, 1)
```

The class inherits from `nn.Module` so that PyTorch knows its weights (for the optimizer and `state_dict`). `n_features` lets the same class work for both exercises (6 and 8 inputs). `output_activation` is None in Exercise 1 and is used in Exercise 2.

The path of x in forward: the path is fixed. Every x passes through all layers in order; the network does not choose a route per sample. Shapes for one batch: (64, 6) → (64, 64) → (64, 32) → (64, 1). Each neuron computes $z = w_1x_1 + \ldots + w_6x_6 + b$, then ReLU applies max(0, z), which sets negative values to zero. Without ReLU, stacked Linear layers would just be one linear regression. The last layer has no activation because the output can be any real number.

Number of parameters (printed output): (6×64 + 64) + (64×32 + 32) + (32×1 + 1) = 448 + 2080 + 33 = **2561**. The ReLUs have no weights. 2561 parameters versus 635 training samples means the model can memorize the data, which is a good reason for early stopping.



Each line in the picture is a real weight of the trained model. Red = positive, blue = negative, darker = larger.

Where do the weights come from? At first they are random: every `nn.Linear` fills them with small numbers between about ±1/√(number of inputs), about ±0.41 for the first layer. With a fixed seed the same numbers are created every time. Randomness is needed because identical weights would make all neurons learn the same thing. During training, for each batch: forward pass and MSE → `loss.backward()` computes each weight's share of the error (the gradient) → `optimizer.step()` moves each weight a little against its gradient: w ← w − 0.01 × gradient, plus momentum.

Step 5 – Training and early stopping

Loss = `nn.MSELoss`, because this is a regression problem and it is the same metric we report. Optimizer = `SGD(lr=0.01, momentum=0.9)`. These two values are not given in the sheet; we chose them. They are common starting values, and other combinations (lr from 0.001 to 0.01, with and without momentum) gave almost the same result (MSE about 0.84 to 0.86). Momentum keeps an average of previous gradients, so the steps are smoother and faster. If the professor asks: "They are standard values. To tune them we would compare a few values on the validation set, not on the test set."

Where each requirement of the sheet is done in the code:

```
# 1) compute and store train and validation loss
loss_fn = nn.MSELoss()
history = {"train": [], "val": []}
val_loss = evaluate_loss(model, val_loader, loss_fn)
history["train"].append(train_loss); history["val"].append(val_loss)
```

```
# 2) keep track of the best validation loss
if val_loss < best_val:
    best_val, best_epoch = val_loss, epoch
    best_weights = copy.deepcopy(model.state_dict())
    epochs_without_improvement = 0

# 3) stop after 10 epochs without improvement
else:
    epochs_without_improvement += 1
if epochs_without_improvement >= patience:
    break

# 4) restore the best weights (after the loop)
model.load_state_dict(best_weights)
```

`best_val` starts at infinity so that the first epoch is always the best so far. `deepcopy` is necessary; without it only a reference would be stored, which changes as training continues. `evaluate_loss` uses `model.eval()` and `torch.no_grad()`, so the weights are not changed and no gradients are computed.

verbose: only a switch for printing progress; it has no effect on the result. True in Exercise 1 (prints epoch 1 and every 10th epoch), False in Exercise 2 (4 models in a row, only the final table matters).

What happened: the best validation MSE was at epoch 23 (0.794). In epochs 24 to 33 it did not improve (e.g. 0.815 at epoch 30), so training stopped at epoch 33 and the weights of epoch 23 were restored. In the same period the training loss still fell from about 0.67 to 0.58, which is the beginning of overfitting. The best weights themselves (2561 numbers) are not printed, only their epoch and loss.

Step 6 – Evaluation on the test set

```
y_pred = predict(model, test_ds.tensors[0])   # all 182 test compounds at once
mse = mean_squared_error(y_test, y_pred)  # likewise mae, r2, pearsonr
```

At this point the model has the weights of epoch 23. `predict` runs only the forward pass with eval and no_grad, and `.numpy().ravel()` turns the (182, 1) output into a (182,) array. The model has never seen y_test, neither in training nor when choosing the epoch, so this is a fair estimate. Pearson comes from scipy because sklearn does not have it.

What the metrics mean:

MSE = 0.862: mean squared error. It penalizes large errors heavily and its unit is LC50 squared. Most useful for comparison: model 0.86 vs. baseline 2.42.

MAE = 0.687: the typical error. On average a prediction is about 0.7 units away from the true value. Because the scale is logarithmic, this means the concentration is off by a factor of about 5.

R^2 = 0.644: the model explains about 64 % of the differences in toxicity between compounds. 1 = perfect, 0 = as good as the mean, negative = worse than the mean.

Pearson r = 0.811: predictions and true values go up and down together in a strong, linear way. But r ignores shifts: if all predictions were 2 units too high, r would stay the same while the MSE would be bad. That is why we report the metrics together.

Plots: in the scatter plot the points lie around y = x. Low-toxicity compounds are slightly overestimated and very toxic ones underestimated (pull towards the mean). The loss plot uses a log scale because the loss in the first epoch is 9.3. The test MSE (0.86) is a bit higher than the best validation MSE (0.79) because the validation set is small and was used to choose the epoch.

4. Exercise 2, step by step

Preprocessing: the same as in Exercise 1, with the prefix h so that the Exercise 1 variables are not overwritten. 20,640 districts with 8 features (income, house age, rooms, population, coordinates). The target is the median house value in units of 100,000 $, from 0.15 to 5.00. Split 14448/2064/4128, scaler fitted on train, batch size 64. The sheet says to re-use the preprocessing from the in-class exercise; we did not have that notebook, so we wrote this standard version.

992 districts with value ≥ 5.0: values above 500k $ were not recorded in this data (1990 census); all of them were set to 5.0 (censoring). About 5 % of the data lies exactly at 5.0 and nothing is above it. This is the basis of the Extra question. The cap is a property of the data, not of real house prices.

Code change for the ≥ 0 constraint: the last layer (w·x + b) can be negative, and the unconstrained model did predict −0.28 once. The only code change:

```
model = RegressionNet(n_features, output_activation=nn.Softplus())
```

Softplus(z) = log(1 + e^z) is always positive: about 0.00005 for z = −10, about 0.69 for z = 0, about 5.007 for z = 5. It is a smooth ReLU with a derivative everywhere and almost linear for large z, so training stays stable. exp is also positive but grows very fast and gives large gradients; that is why it stopped earlier (epoch 17) with a higher MSE (0.308). ReLU as output would also work, but it has zero gradient for negative z, so the output can get stuck at zero.

The MSE with Softplus (0.293) is almost the same as without constraint (0.290), because only 1 of 4128 predictions was negative. The benefit of the constraint is that the predictions make sense.

Extra, upper limit 500k:

```
class ScaledSigmoid(nn.Module):
    def forward(self, x):
        return self.upper * torch.sigmoid(x)   # always between 0 and 5
```

sigmoid lies between 0 and 1, multiplying by 5 gives the range 0 to 5. This layer has no weights. It was trained with the same `fit_and_test` function. Result: MSE = 0.291, predictions between 0.41 and 5.00, no prediction above 5 (the unconstrained model had 37). Drawback: sigmoid is flat near its limits, so gradients get small there.

Why is 5.0 hard-coded? Because the sheet gives the number ("limited to 500k$"). In real work, constraints come from domain knowledge, the dataset documentation and EDA on the training data. A more general version is `upper = yh_train.max()`, using only train and never test.

5. Likely questions from the professor

Why MSE as loss? It is the standard regression loss, smooth and differentiable, it penalizes large errors more, and it is the reported metric. Alternatives: MAE or Huber loss with many outliers.

Why fit the scaler only on train? To avoid data leakage. The test set must stay unseen, like future data.

How does early stopping work, and why patience? The best validation loss and its weights are kept. 10 epochs without improvement means stop, then the best weights are restored. Patience is needed because the validation loss is noisy.

What if the best weights were not restored? The model would keep the weights of epoch 33, which are slightly overfitted.

Why is the test MSE higher than the validation MSE? The validation set has only 91 samples and was used to choose the epoch, so it is slightly optimistic. The test set played no role in any decision.

Is R^2 = 0.64 good? Much better than the baseline and in the range of the original study of this dataset. Six simple descriptors cannot explain all toxicity mechanisms.

Difference between r and R^2? r only measures the linear relationship and ignores shifts. R^2 measures the actual error relative to the mean. Here r^2 = 0.66 and R^2 = 0.64.

Why no activation in the last layer in Exercise 1? The output can be any real number.

Where do lr and momentum come from? The sheet gives no values. They are common values and other combinations gave similar results. Tuning should be done on the validation set.

What if the learning rate is too large or too small? Too large: the loss oscillates or explodes. Too small: training is very slow.

Why is y reshaped to (n, 1)? The network output is (batch, 1). With (batch,), MSELoss broadcasts to a matrix and the loss is wrong.

Exercise 2: what changed in the code? Only `output_activation=nn.Softplus()`. Test MSE = 0.293, no negative predictions.

Softplus or exp? Why not ReLU? Softplus is more stable; exp grows too fast; ReLU has zero gradient for negative inputs.

How was the Extra done, and what is the drawback? With 5·sigmoid: MSE = 0.291 and no prediction above 5. Drawback: small gradients near the limits. The cap is a property of this dataset only.

Reproducibility? Fixed seeds for numpy, torch and the DataLoader shuffling, `random_state=42` in the split, and `torch.manual_seed` before building each model.

How could the model be improved? Hyperparameter search on the validation set, weight decay or dropout, Adam, and cross-validation because the dataset is small.